

MODUL PRAKTIKUM

KRIPTOGRAFI DAN KEAMANAN DATA

Disusun untuk Mendukung Format Penulisan Laporan Praktikum

Software: OpenSSL, Python, Wireshark, Browser

Waktu: 30–60 menit per praktikum

Prasyarat: Baca buku teori kriptografi

Departemen Teknik Informatika Tahun Akademik 2025/2026

DAFTAR ISI

Praktikum 1: Konsep Enkripsi dan Dekripsi

- > A. Judul
- > B. Tujuan
- > C. Teori Singkat
- > D. Langkah-langkah
- > E. Hasil Praktikum (Dasar)
- > F. Persoalan Sedang
- > G. Persoalan Susah
- > H. Kesimpulan

Praktikum 2: Algoritma Kriptografi Simetris dan Asimetris

- > A. Judul
- > B. Tujuan
- > C. Teori Singkat
- > D. Langkah-langkah
- > E. Hasil Praktikum (Dasar)
- > F. Persoalan Sedang
- > G. Persoalan Susah
- > H. Kesimpulan

Praktikum 3: Digital Signature dan Hashing

- > A. Judul
- > B. Tujuan
- > C. Teori Singkat
- > D. Langkah-langkah
- > E. Hasil Praktikum (Dasar)

- > F. Persoalan Sedang
- > G. Persoalan Susah
- > H. Kesimpulan

Praktikum 4: Implementasi Kriptografi dalam Keamanan Siber

- > A. Judul
- > B. Tujuan
- > C. Teori Singkat
- > D. Langkah-langkah
- > E. Hasil Praktikum (Dasar)
- > F. Persoalan Sedang
- > G. Persoalan Susah
- > H. Kesimpulan

Ringkasan Perintah

Tips Mengerjakan Laporan Praktikum

Ketentuan Pengumpulan

- > Format Laporan
- > Format Nama File
- > Pengumpulan

Praktikum 1: Konsep Enkripsi dan Dekripsi

Durasi: 30 menit | Tingkat Kesulitan: Sangat Mudah

A. Judul

Praktikum Konsep Enkripsi dan Dekripsi: Simulasi Caesar Cipher dan Perubahan Plaintext menjadi Ciphertext

B. Tujuan

1. Memahami konsep dasar enkripsi dan dekripsi dalam keamanan data
2. Memahami fungsi key (kunci) dalam proses kriptografi
3. Melakukan enkripsi sederhana menggunakan Caesar Cipher secara manual dan berbasis Python
4. Melakukan dekripsi pesan sederhana dari ciphertext menjadi plaintext
5. Memahami perubahan bentuk data dari plaintext menjadi ciphertext

C. Teori Singkat

Enkripsi adalah proses mengubah data asli (plaintext) menjadi bentuk yang tidak dapat dibaca (ciphertext) menggunakan algoritma tertentu dan kunci rahasia. Proses ini bertujuan untuk melindungi kerahasiaan informasi dari pihak yang tidak berwenang. Enkripsi merupakan salah satu pilar utama dalam ilmu kriptografi yang telah digunakan sejak zaman dahulu hingga era digital saat ini.

Dekripsi adalah kebalikan dari enkripsi, yaitu proses mengubah ciphertext kembali menjadi plaintext yang dapat dibaca menggunakan kunci yang sesuai. Tanpa kunci yang benar, proses dekripsi tidak dapat dilakukan dengan mudah sehingga keamanan data tetap terjaga.

Caesar Cipher adalah salah satu metode enkripsi tertua dan paling sederhana yang ditemukan oleh Julius Caesar. Metode ini bekerja dengan cara menggeser setiap huruf dalam plaintext sejumlah posisi tertentu pada alfabet. Misalnya, dengan geseran (shift) 3, huruf A menjadi D, B menjadi E, dan seterusnya.

Plaintext adalah data asli yang belum dienkripsi dan masih dalam bentuk yang dapat dibaca.

Ciphertext adalah hasil dari proses enkripsi yang berupa data teracak dan tidak dapat dibaca tanpa proses dekripsi.

Key (Kunci) adalah parameter rahasia yang digunakan dalam algoritma enkripsi dan dekripsi. Keamanan sistem kriptografi sangat bergantung pada kerahasiaan

dan kekuatan kunci yang digunakan. Konsep ini juga dikenal sebagai confidentiality, yaitu jaminan bahwa informasi hanya dapat diakses oleh pihak yang berwenang.

D. Langkah-langkah

a. Enkripsi Manual (Caesar Cipher)

1. Siapkan plaintext: "HELLO"
2. Tentukan key (geseran): 3
3. Geser setiap huruf sebanyak 3 posisi ke depan pada alfabet
4. Catat hasil ciphertext yang diperoleh

Huruf Asli	Geser +3	Hasil
H	I, J, K	K
E	F, G, H	H
L	M, N, O	O
L	M, N, O	O
O	P, Q, R	R

Hasil: HELLO → KHOOR

b. Dekripsi Manual

Dekripsi dilakukan dengan cara menggeser setiap huruf ciphertext KHOOR sebesar 3 posisi ke belakang pada alfabet, sehingga menghasilkan kembali plaintext HELLO.

c. Enkripsi Menggunakan Python

```
text = "HELLO"
shift = 3
hasil = ""
for char in text:
    hasil += chr((ord(char) - 65 + shift) % 26 + 65)
print("Ciphertext:", hasil)
# Output: Ciphertext: KHOOR
```

d. Dekripsi Menggunakan Python

```
text = "KHOOR"
shift = 3
hasil = ""
for char in text:
    hasil += chr((ord(char) - 65 - shift) % 26 + 65)
```

```
print("Plaintext:", hasil)
# Output: Plaintext: HELLO
```

e. Simulasi dengan Beberapa Plaintext

Plaintext	Key (Geser)	Ciphertext
DATA	2	FCVC
AKMIL	4	EOPMP
CYBER	1	DZCFS
SISKO	5	XNXQT

E. Hasil Praktikum (Dasar)

1. Apa yang dimaksud dengan enkripsi? Jelaskan dengan kata-kata Anda sendiri.

Jawaban:

.....

2. Apa fungsi key (kunci) dalam proses enkripsi dan dekripsi?

Jawaban:

.....

3. Lakukan enkripsi manual untuk plaintext "SECURE" dengan key = 2. Berapa ciphertext yang dihasilkan?

Jawaban:

.....

4. Dekripsikan ciphertext "LIPPS" dengan key = 3. Berapa plaintext yang diperoleh?

Jawaban:

.....

5. Jalankan program Python enkripsi di atas. Sertakan screenshot hasil output.

Jawaban:

.....

Screenshot:

.....

6. Jalankan program Python dekripsi di atas. Sertakan screenshot hasil output.

Jawaban:

.....

Screenshot:

.....

7. Ubah plaintext dalam program Python menjadi nama Anda (huruf kapital). Sertakan hasil enkripsinya.

Jawaban:

.....

8. Apa yang terjadi jika key yang digunakan untuk dekripsi berbeda dengan key enkripsi?

Jawaban:

.....

F. Persoalan Sedang — Caesar Cipher Lanjutan

Durasi: 20 menit | Tingkat Kesulitan: Sedang

Buatlah program Python Caesar Cipher yang mampu menangani huruf kapital (A-Z), huruf kecil (a-z), serta karakter spasi dan tanda baca (dienkripsi apa adanya, tidak diubah). Key geser dinamis (1–25) diinput oleh user.

Tugas:

1. Enkripsi kalimat berikut dengan key = 7: "Keamanan Data 2026!"

Jawaban:

.....

2. Dekripsikan kembali ciphertext yang dihasilkan untuk membuktikan kebenaran program.

Jawaban:

.....

Screenshot:

.....

3. Jelaskan dalam 3–5 baris: Mengapa Caesar Cipher dengan key statis rentan terhadap serangan brute-force?

Jawaban:

.....

Referensi Kode:

```
def caesar_cipher_advanced(text, shift, mode='encrypt'):
    result = ""
    for char in text:
        if char.isupper():
            base = ord('A')
            if mode == 'encrypt':
                result += chr((ord(char) - base + shift) % 26 + base)
            else:
                result += chr((ord(char) - base - shift) % 26 + base)
        elif char.islower():
            base = ord('a')
            if mode == 'encrypt':
                result += chr((ord(char) - base + shift) % 26 + base)
            else:
                result += chr((ord(char) - base - shift) % 26 + base)
        else:
            result += char # Spasi dan tanda baca tidak diubah
    return result

# Contoh penggunaan
plaintext = "Keamanan Data 2026!"
shift = 7
ciphertext = caesar_cipher_advanced(plaintext, shift, 'encrypt')
print("Ciphertext:", ciphertext)
decrypted = caesar_cipher_advanced(ciphertext, shift, 'decrypt')
print("Plaintext:", decrypted)
```

G. Persoalan Susah — Cryptanalysis: Serangan Brute-Force

Durasi: 30 menit | Tingkat Kesulitan: Susah

Anda mendapatkan ciphertext berikut tanpa mengetahui key-nya: "ZHOFRPH
WR FUBSWRJUDSKB"

Tugas:

1. Buat program Python yang mencoba semua kemungkinan key (1–25) secara otomatis.


```

        decrypted += char

    # Hitung skor berdasarkan kemunculan kata umum
    score = sum(1 for word in common_words if word in decrypted)
    candidates.append((key, decrypted, score))

    # Urutkan berdasarkan skor tertinggi
    candidates.sort(key=lambda x: x[2], reverse=True)
    return candidates[:5] # Top 5 kandidat

ciphertext = "ZHOFRPH WR FUBSWRJUDSKB"
results = brute_force_caesar(ciphertext)
for key, text, score in results:
    print(f"Key {key}: {text} (Score: {score})")

```

H. Kesimpulan

- Tuliskan kesimpulan Anda tentang konsep enkripsi dan dekripsi berdasarkan praktikum yang telah dilakukan.
- Tuliskan kesimpulan Anda tentang penggunaan Caesar Cipher.
- Tuliskan hal-hal yang Anda pelajari dari praktikum ini.
- Tuliskan kendala yang Anda hadapi dan bagaimana cara mengatasinya.
- Tuliskan refleksi Anda tentang pentingnya memahami kelemahan algoritma sederhana (Caesar Cipher) dalam konteks keamanan modern.

Praktikum 2: Algoritma Kriptografi Simetris dan Asimetris

Durasi: 45 menit | Tingkat Kesulitan: Menengah

A. Judul

Implementasi AES dan RSA Menggunakan OpenSSL

B. Tujuan

6. Melakukan enkripsi dan dekripsi file menggunakan algoritma AES-256-CBC
7. Membuat pasangan kunci RSA (public key dan private key) menggunakan OpenSSL
8. Melakukan enkripsi dan dekripsi pesan menggunakan RSA
9. Memahami perbedaan konsep kriptografi simetris dan asimetris
10. Menganalisis kelebihan dan kekurangan masing-masing metode kriptografi

C. Teori Singkat

Kriptografi Simetris menggunakan satu kunci yang sama untuk proses enkripsi dan dekripsi. Kunci ini harus dirahasiakan dan hanya diketahui oleh pengirim dan penerima pesan. Algoritma simetris yang paling umum digunakan adalah AES (Advanced Encryption Standard), khususnya varian AES-256 yang menggunakan kunci berukuran 256 bit. AES merupakan standar enkripsi yang diadopsi oleh pemerintah Amerika Serikat dan banyak organisasi di seluruh dunia karena keamanan dan efisiensinya.

Kriptografi Asimetris menggunakan sepasang kunci: public key (kunci publik) yang dapat dibagikan kepada siapa saja, dan private key (kunci privat) yang harus dirahasiakan. Pesan yang dienkripsi dengan public key hanya dapat didekripsi dengan private key yang sesuai, dan sebaliknya. RSA (Rivest-Shamir-Adleman) adalah salah satu algoritma asimetris paling terkenal yang banyak digunakan dalam protokol keamanan internet seperti SSL/TLS, SSH, dan HTTPS.

OpenSSL adalah toolkit open-source yang menyediakan implementasi berbagai protokol kriptografi dan fungsi utilitas keamanan. OpenSSL digunakan secara luas di server web, sistem operasi, dan aplikasi untuk mengamankan komunikasi data melalui internet.

D. Langkah-langkah

A. Enkripsi File dengan AES-256-CBC

1. Buat file teks bernama `rahasia.txt` dengan isi:

```
echo "Data Rahasia Siber" > rahasia.txt
```

- a) Enkripsi file menggunakan perintah:

```
openssl enc -aes-256-cbc -salt -in rahasia.txt -out rahasia.enc
```

Perintah ini akan meminta Anda memasukkan passphrase (password) untuk enkripsi. Masukkan password yang aman dan konfirmasi.

- b) Dekripsi file menggunakan perintah:

```
openssl enc -d -aes-256-cbc -in rahasia.enc -out hasil.txt
```

- c) Verifikasi hasil dekripsi:

```
cat hasil.txt
```

B. Membuat Pasangan Kunci RSA

1. Buat private key RSA 2048-bit:

```
openssl genrsa -out private.pem 2048
```

2. Ekstrak public key dari private key:

```
openssl rsa -in private.pem -pubout -out public.pem
```

C. Enkripsi dan Dekripsi Pesan dengan RSA

1. Buat file `pesan.txt` dengan isi:

```
echo "Pesan Rahasia untuk AKMIL" > pesan.txt
```

2. Enkripsi pesan dengan public key:

```
openssl rsautl -encrypt -pubin -inkey public.pem -in pesan.txt -out pesan.enc
```

3. Dekripsi pesan dengan private key:

```
openssl rsautl -decrypt -inkey private.pem -in pesan.enc -out hasil_dekripsi.txt
```

4. Verifikasi hasil dekripsi:

```
cat hasil_dekripsi.txt
```

D. Tabel Perbandingan

Isilah tabel perbandingan berikut berdasarkan hasil praktikum dan pemahaman Anda:

Aspek	Simetris (AES)	Asimetris (RSA)
Jumlah Key		
Kecepatan Proses		
Distribusi Key		
Keamanan		
Penggunaan Umum		

E. Hasil Praktikum (Dasar)

1. Apa perbedaan utama antara kriptografi simetris dan asimetris?

Jawaban:

.....

2. Sertakan screenshot hasil enkripsi AES-256-CBC.

Jawaban:

.....

Screenshot:

.....

3. Sertakan screenshot hasil dekripsi AES-256-CBC.

Jawaban:

.....

Screenshot:

.....

4. Sertakan screenshot pembuatan pasangan kunci RSA.

Jawaban:

.....

Screenshot:

.....

5. Sertakan screenshot enkripsi dan dekripsi pesan RSA.

Jawaban:

.....

Screenshot:

.....

6. Apa yang terjadi jika Anda mencoba mendekripsi file AES dengan password yang salah?

Jawaban:

.....

7. Mengapa RSA lebih lambat dibandingkan AES? Jelaskan.

Jawaban:

.....

8. Dalam situasi apa sebaiknya menggunakan AES, dan kapan menggunakan RSA?

Jawaban:

.....

F. Persoalan Sedang — Hybrid Encryption Shell Script

Durasi: 30 menit | Tingkat Kesulitan: Sedang

Buatlah script Bash otomatis (.sh) yang melakukan proses hybrid encryption lengkap: generate pasangan kunci RSA 4096-bit, buat file pesan_rahasia.txt, enkripsi file menggunakan AES-256-CBC dengan password acak (generate otomatis), enkripsi password AES menggunakan public key RSA, dan buat script dekripsi terpisah.

Tugas:

1. Buat script encrypt.sh yang mengotomatisasi seluruh proses enkripsi hybrid.

Jawaban:

.....

2. Buat script decrypt.sh yang melakukan dekripsi password AES menggunakan private key, lalu dekripsi file menggunakan password tersebut.

Jawaban:

.....

3. Sertakan screenshot hasil eksekusi script enkripsi dan dekripsi.

Jawaban:

.....

Screenshot:

.....

4. Jelaskan mengapa konsep hybrid encryption (AES + RSA) lebih efisien untuk file besar dibandingkan menggunakan RSA langsung untuk mengenkripsi seluruh file.

Jawaban:

.....

Referensi Script:

```
#!/bin/bash
# encrypt.sh - Hybrid Encryption Script

# 1. Generate RSA 4096-bit key pair
openssl genrsa -out private.pem 4096 2>/dev/null
openssl rsa -in private.pem -pubout -out public.pem 2>/dev/null

# 2. Create secret message
echo "Pesan Rahasia Hybrid Encryption" > pesan_rahasia.txt

# 3. Generate random AES password
AES_PASS=$(openssl rand -base64 32)
echo "$AES_PASS" > aes_password.txt

# 4. Encrypt file with AES-256-CBC
openssl enc -aes-256-cbc -salt -pass pass:"$AES_PASS" -in
pesan_rahasia.txt -out pesan_rahasia.enc

# 5. Encrypt AES password with RSA public key
openssl pkeyutl -encrypt -in aes_password.txt -inkey public.pem -
pubin -out aes_key.enc

# 6. Remove plaintext password file
rm aes_password.txt
```

```

echo "Enkripsi selesai! File: pesan_rahasia.enc, aes_key.enc"

# decrypt.sh - Hybrid Decryption Script
# openssl pkeyutl -decrypt -in aes_key.enc -inkey private.pem -out
aes_password.txt
# AES_PASS=$(cat aes_password.txt)
# openssl enc -d -aes-256-cbc -pass pass:"$AES_PASS" -in
pesan_rahasia.enc -out hasil.txt
# cat hasil.txt

```

G. Persoalan Susah — Benchmark Hybrid Encryption untuk File Besar

Durasi: 45 menit | Tingkat Kesulitan: Susah

Implementasikan sistem enkripsi file besar (> 50 MB) menggunakan konsep Hybrid Cryptosystem (AES-256-CBC untuk enkripsi data, RSA-4096 untuk enkripsi session key). Ukur dan bandingkan waktu eksekusi.

Tugas:

1. Buat program Python lengkap dengan mekanisme enkripsi: Generate random AES key → enkripsi file dengan AES → enkripsi AES key dengan RSA public key → output: file.enc + key.enc.

Jawaban:

.....

2. Buat mekanisme dekripsi: Dekripsi AES key dengan RSA private key → dekripsi file dengan AES key.

Jawaban:

.....

3. Ukur dan bandingkan waktu eksekusi untuk file 10MB, 50MB, dan 100MB pada 3 skenario: Enkripsi murni AES-256-CBC, Enkripsi murni RSA (untuk ukuran kecil), dan Hybrid Encryption (AES + RSA).

Jawaban:

.....

4. Buat tabel perbandingan dan grafik sederhana (menggunakan Python matplotlib) yang menunjukkan perbedaan waktu dan ukuran output.

Jawaban:

.....

Screenshot / Hasil:

.....

5. Analisis: Mengapa RSA tidak cocok untuk enkripsi data berukuran besar dari segi matematis dan komputasional?

Jawaban:

.....

Referensi Kode:

```
import time
import os
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

def generate_test_file(size_mb, filename):
    """Generate test file dengan ukuran tertentu"""
    with open(filename, 'wb') as f:
        f.write(os.urandom(size_mb * 1024 * 1024))

def hybrid_encrypt(file_path, public_key_path):
    # Generate random AES key
    aes_key = get_random_bytes(32)

    # Read public key
    with open(public_key_path, 'rb') as f:
        public_key = RSA.import_key(f.read())

    # Encrypt file with AES
    cipher_aes = AES.new(aes_key, AES.MODE_CBC)
    with open(file_path, 'rb') as f:
        plaintext = f.read()
    ciphertext = cipher_aes.encrypt(pad(plaintext, AES.block_size))

    # Encrypt AES key with RSA
    cipher_rsa = PKCS1_OAEP.new(public_key)
    enc_aes_key = cipher_rsa.encrypt(aes_key)

    return ciphertext, cipher_aes.iv, enc_aes_key

# Benchmark script
```

```
sizes = [10, 50, 100]
for size in sizes:
    generate_test_file(size, f'test_{size}mb.bin')
    start = time.time()
    # ... proses enkripsi ...
    elapsed = time.time() - start
    print(f"Size {size}MB: {elapsed:.2f} detik")
```

H. Kesimpulan

- Tuliskan kesimpulan Anda tentang kriptografi simetris (AES).
- Tuliskan kesimpulan Anda tentang kriptografi asimetris (RSA).
- Tuliskan perbandingan kelebihan dan kekurangan kedua metode.
- Tuliskan hal-hal yang Anda pelajari dari praktikum ini.
- Tuliskan refleksi Anda tentang pentingnya hybrid encryption dalam sistem keamanan modern.

Praktikum 3: Digital Signature dan Hashing

Durasi: 35 menit | Tingkat Kesulitan: Menengah

A. Judul

Verifikasi Integritas Data dan Autentikasi Digital

B. Tujuan

11. Memahami konsep fungsi hash dan penggunaannya dalam keamanan data
12. Membuat hash SHA-256 dari sebuah file menggunakan command line
13. Memverifikasi integritas data dengan membandingkan hash sebelum dan sesudah modifikasi
14. Membuat digital signature menggunakan OpenSSL
15. Memverifikasi digital signature dan memahami perannya dalam autentikasi

C. Teori Singkat

Hashing adalah proses mengubah data input dengan panjang berapa pun menjadi output berupa string karakter tetap yang disebut hash value atau message digest. Fungsi hash bersifat satu arah (one-way function), artinya hash value tidak dapat dikembalikan menjadi data asli. Perubahan kecil pada input akan menghasilkan perubahan drastis pada hash value, yang dikenal sebagai efek avalanche.

SHA-256 (Secure Hash Algorithm 256-bit) adalah fungsi hash kriptografi yang menghasilkan hash value berukuran 256 bit (64 karakter heksadesimal). SHA-256 merupakan bagian dari keluarga SHA-2 yang dikembangkan oleh NSA dan saat ini merupakan standar hashing yang paling banyak digunakan di dunia, termasuk dalam teknologi blockchain dan sertifikat digital.

Digital Signature adalah mekanisme kriptografi yang menggunakan kombinasi hashing dan enkripsi asimetris untuk memastikan autentikasi (keaslian pengirim), integritas data (data tidak diubah), dan non-repudiation (pengirim tidak dapat menyangkal telah mengirim pesan). Digital signature dibuat dengan mengenkripsi hash dari pesan menggunakan private key pengirim, dan diverifikasi menggunakan public key pengirim.

Integritas Data adalah jaminan bahwa data tidak diubah, dihapus, atau dimanipulasi secara tidak sah selama proses penyimpanan atau transmisi. Verifikasi integritas dilakukan dengan membandingkan hash value data yang diterima dengan hash value data asli.

D. Langkah-langkah

A. Membuat Hash SHA-256

1. Buat file bernama laporan.txt:

```
echo "Ini adalah laporan praktikum keamanan siber." > laporan.txt
```

2. Buat hash SHA-256 dari file tersebut:

```
sha256sum laporan.txt
```

Catat hash value yang dihasilkan. Hash value ini adalah sidik jari digital unik dari file Anda.

B. Verifikasi Integritas Data

1. Modifikasi isi file laporan.txt (tambahkan satu karakter saja):

```
echo "Ini adalah laporan praktikum keamanan siber!" > laporan.txt
```

2. Hitung ulang hash SHA-256:

```
sha256sum laporan.txt
```

Bandingkan hash value baru dengan hash value sebelumnya. Perhatikan bahwa perubahan satu karakter saja menghasilkan hash yang sangat berbeda (efek avalanche).

C. Membuat Digital Signature

1. Gunakan private key RSA yang dibuat pada Praktikum 2. Buat digital signature:

```
openssl dgst -sha256 -sign private.pem -out sign.bin laporan.txt
```

File sign.bin adalah digital signature dari laporan.txt yang dibuat menggunakan private key Anda.

D. Verifikasi Digital Signature

1. Verifikasi signature dengan public key:

```
openssl dgst -sha256 -verify public.pem -signature sign.bin  
laporan.txt
```

Jika file tidak diubah, output akan menampilkan "Verified OK".

Sekarang modifikasi file dan coba verifikasi ulang:

```
echo "Data yang dimodifikasi" > laporan.txt
openssl dgst -sha256 -verify public.pem -signature sign.bin
laporan.txt
```

Output akan menampilkan "Verification Failure" karena file telah diubah sehingga signature tidak lagi valid.

E. Hasil Praktikum (Dasar)

1. Apa yang dimaksud dengan hashing? Mengapa hashing bersifat satu arah?

Jawaban:

.....

2. Sertakan screenshot hash SHA-256 dari file asli.

Jawaban:

.....

Screenshot:

.....

3. Sertakan screenshot hash SHA-256 dari file yang telah dimodifikasi. Apa yang Anda amati?

Jawaban:

.....

Screenshot:

.....

4. Sertakan screenshot pembuatan digital signature.

Jawaban:

.....

Screenshot:

.....

5. Sertakan screenshot verifikasi digital signature (Verified OK).

Jawaban:

.....
Screenshot:

.....
6. Sertakan screenshot verifikasi digital signature setelah file dimodifikasi (Verification Failure).

Jawaban:

.....
Screenshot:

.....
7. Mengapa digital signature penting dalam komunikasi digital? Jelaskan tiga fungsinya.

Jawaban:

.....
8. Apa perbedaan antara hashing dan enkripsi? Berikan contoh penggunaan masing-masing.

Jawaban:

F. Persoalan Sedang — Sistem Integritas Folder dengan Manifest

Durasi: 30 menit | Tingkat Kesulitan: Sedang

Buatlah sistem verifikasi integritas untuk sebuah direktori/folder. Script akan menelusuri semua file dalam direktori, menghitung hash SHA-256 untuk setiap file, menyimpan hasilnya dalam file integrity_manifest.txt, dan membuat digital signature dari manifest.

Tugas:

1. Buat script (Python/Bash) yang menelusuri semua file dalam sebuah direktori, menghitung hash SHA-256 untuk setiap file, dan menyimpan hasilnya dalam file integrity_manifest.txt dengan format: [nama_file] [hash_sha256].

Jawaban:

.....

2. Buat digital signature dari file integrity_manifest.txt menggunakan private key RSA (dari Praktikum 2).

Jawaban:

.....

3. Buat script verifikasi terpisah yang memeriksa signature manifest menggunakan public key, membandingkan ulang hash file-file saat ini dengan hash dalam manifest, dan melaporkan file mana yang berubah, ditambah, atau dihapus.

Jawaban:

.....

4. Simulasikan dengan mengubah isi salah satu file, menambah file baru, dan menghapus file lama. Sertakan screenshot hasil verifikasi yang mendeteksi perubahan tersebut.

Jawaban:

.....

Screenshot:

.....

Referensi Script:

```
#!/bin/bash
# create_manifest.sh

DIRECTORY="./test_folder"
MANIFEST="integrity_manifest.txt"

# Generate manifest
echo "# Integrity Manifest - Generated $(date)" > $MANIFEST
echo "# Format: filename hash" >> $MANIFEST

for file in "$DIRECTORY"/*; do
    if [ -f "$file" ]; then
        filename=$(basename "$file")
        hash=$(sha256sum "$file" | awk '{print $1}')
        echo "$filename $hash" >> $MANIFEST
    fi
done
```

```

done

echo "Manifest created: $MANIFEST"

# Sign manifest
openssl dgst -sha256 -sign private.pem -out manifest.sig $MANIFEST

# verify_manifest.sh
# 1. Verify signature
openssl dgst -sha256 -verify public.pem -signature manifest.sig
$MANIFEST

# 2. Compare current hashes with manifest
while read -r line; do
    if [[ $line != \#* ]]; then
        filename=$(echo "$line" | awk '{print $1}')
        expected_hash=$(echo "$line" | awk '{print $2}')
        if [ -f "$DIRECTORY/$filename" ]; then
            current_hash=$(sha256sum "$DIRECTORY/$filename" | awk
'{print $1}')
            if [ "$expected_hash" != "$current_hash" ]; then
                echo "MODIFIED: $filename"
            fi
        else
            echo "DELETED: $filename"
        fi
    fi
done < $MANIFEST

```

G. Persoalan Susah — Blockchain Sederhana: Rantai Hash

Durasi: 45 menit | Tingkat Kesulitan: Susah

Implementasikan konsep Hash Chain (dasar blockchain) untuk memahami integritas berkelanjutan. Buat 5 blok data berturut-turut yang saling terhubung melalui hash.

Tugas:

1. Buat 5 blok data berturut-turut dengan struktur setiap blok: block_id (1–5), timestamp, data_transaksi (string bebas), prev_hash (hash SHA-256 dari blok sebelumnya; blok 1/genesis block menggunakan "0"), current_hash (hash SHA-256 dari gabungan block_id + timestamp + data_transaksi + prev_hash).

Jawaban:

.....

2. Implementasikan dalam Python. Tampilkan seluruh rantai blok dengan hash masing-masing.

Jawaban:

.....

3. Simulasi Serangan: Ubah data_transaksi pada blok ke-3 secara manual di dalam kode/script.

Jawaban:

.....

4. Buat fungsi verifikasi rantai yang memeriksa apakah setiap current_hash dan prev_hash masih konsisten.

Jawaban:

.....

5. Tunjukkan bahwa perubahan di blok ke-3 menyebabkan blok ke-3, 4, dan 5 menjadi tidak valid.

Jawaban:

.....

6. Jelaskan hubungan konsep ini dengan teknologi blockchain nyata dan mengapa blockchain dianggap tamper-evident (mudah terdeteksi jika diubah).

Jawaban:

.....

Referensi Kode:

```
import hashlib
import time
import json

class Block:
    def __init__(self, block_id, timestamp, data_transaksi,
prev_hash):
        self.block_id = block_id
        self.timestamp = timestamp
        self.data_transaksi = data_transaksi
        self.prev_hash = prev_hash
        self.current_hash = self.calculate_hash()
```

```

    def calculate_hash(self):
        block_string = (str(self.block_id) + str(self.timestamp) +
                        str(self.data_transaksi) +
str(self.prev_hash))
        return hashlib.sha256(block_string.encode()).hexdigest()

    def to_dict(self):
        return {
            'block_id': self.block_id,
            'timestamp': self.timestamp,
            'data_transaksi': self.data_transaksi,
            'prev_hash': self.prev_hash,
            'current_hash': self.current_hash
        }

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(1, time.time(), "Genesis Block", "0")

    def add_block(self, data_transaksi):
        prev_block = self.chain[-1]
        new_block = Block(len(self.chain) + 1, time.time(),
                           data_transaksi, prev_block.current_hash)
        self.chain.append(new_block)

    def verify_chain(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i-1]

            if current.current_hash != current.calculate_hash():
                return False, f"Hash blok {current.block_id} tidak
valid!"

            if current.prev_hash != previous.current_hash:
                return False, f"Prev_hash blok {current.block_id}
tidak cocok!"

        return True, "Rantai valid!"

# Penggunaan
bc = Blockchain()
bc.add_block("Transaksi A: Transfer 100 BTC")
bc.add_block("Transaksi B: Transfer 50 BTC")
bc.add_block("Transaksi C: Transfer 200 BTC")
bc.add_block("Transaksi D: Transfer 75 BTC")

```

```
# Tampilkan rantai
for block in bc.chain:
    print(json.dumps(block.to_dict(), indent=2))

# Verifikasi
print(bc.verify_chain())

# Simulasi serangan: ubah data blok 3
bc.chain[2].data_transaksi = "Transaksi B: Transfer 9999 BTC"
print(bc.verify_chain()) # Akan gagal!
```

H. Kesimpulan

- Tuliskan kesimpulan Anda tentang fungsi hash dan SHA-256.
- Tuliskan kesimpulan Anda tentang digital signature.
- Tuliskan bagaimana hashing dan digital signature bekerja sama untuk menjamin keamanan data.
- Tuliskan hal-hal yang Anda pelajari dari praktikum ini.
- Tuliskan refleksi Anda tentang hubungan konsep hash chain dengan teknologi blockchain modern.

Praktikum 4: Implementasi Kriptografi dalam Keamanan Siber

Durasi: 40 menit | Tingkat Kesulitan: Menengah

A. Judul

Analisis HTTPS, Wireshark, dan Password Encryption

B. Tujuan

16. Memahami implementasi kriptografi pada protokol HTTPS
17. Menganalisis sertifikat digital pada website menggunakan browser
18. Menggunakan Wireshark untuk menganalisis paket jaringan terenkripsi (TLS)
19. Melakukan enkripsi file menggunakan ZIP dengan password
20. Memahami bagaimana kriptografi diterapkan dalam kehidupan sehari-hari

C. Teori Singkat

HTTPS (Hypertext Transfer Protocol Secure) adalah versi aman dari HTTP yang menggunakan protokol SSL (Secure Sockets Layer) atau TLS (Transport Layer Security) untuk mengenkripsi komunikasi antara browser dan server. HTTPS memastikan bahwa data yang ditransmisikan tidak dapat disadap atau dimodifikasi oleh pihak ketiga. Saat ini, hampir semua website modern menggunakan HTTPS sebagai standar keamanan.

Sertifikat Digital adalah dokumen elektronik yang mengikat identitas sebuah entitas (seperti website atau organisasi) dengan kunci publik. Sertifikat digital diterbitkan oleh Certificate Authority (CA) yang tepercaya dan berisi informasi seperti nama domain, tanggal berlaku, kunci publik, dan identitas CA penerbit. Browser memverifikasi sertifikat digital untuk memastikan bahwa koneksi ke website tersebut aman dan otentik.

Wireshark adalah alat analisis protokol jaringan open-source yang memungkinkan pengguna untuk menangkap dan memeriksa paket data secara real-time. Dengan Wireshark, kita dapat mengamati proses TLS Handshake, pertukaran sertifikat, dan data yang terenkripsi. Pada koneksi HTTPS, Wireshark hanya dapat melihat paket terenkripsi sehingga konten data tidak dapat dibaca tanpa kunci sesi.

Password Encryption adalah teknik melindungi file dan arsip dengan menggunakan password. Enkripsi password banyak digunakan pada file arsip ZIP, RAR, dan PDF untuk menjaga kerahasiaan konten. Meskipun bukan

metode kriptografi paling kuat, enkripsi password tetap menjadi lapisan keamanan dasar yang penting dan mudah diimplementasikan.

D. Langkah-langkah

A. Analisis HTTPS pada Browser

1. Buka browser dan akses sebuah website yang menggunakan HTTPS (contoh: <https://www.google.com>)
2. Perhatikan ikon gembok (lock) pada address bar browser
3. Klik ikon gembok dan pilih opsi untuk melihat detail sertifikat
4. Amati informasi berikut pada sertifikat:
 - a) Nama domain (Common Name / Subject)
 - a) Certificate Authority (CA) penerbit
 - a) Tanggal berlaku (Valid From - Valid To)
 - a) Algoritma dan panjang kunci publik
 - a) Penggunaan ekstensi (Key Usage, Extended Key Usage)

B. Analisis Paket dengan Wireshark

1. Buka Wireshark dan mulai capture pada interface jaringan aktif
2. Filter paket dengan mengetikkan: tls atau ssl pada filter bar
3. Akses sebuah website HTTPS menggunakan browser
4. Amati proses TLS Handshake pada Wireshark, perhatikan:
 - a) Client Hello
 - a) Server Hello
 - a) Certificate Exchange
 - a) Key Exchange
 - a) Encrypted Application Data

C. Simulasi Komunikasi Terenkripsi

1. Bandingkan capture paket antara website HTTP (tidak aman) dan HTTPS (aman)
2. Amati perbedaan data yang terlihat pada masing-masing protokol
3. Pada HTTP, data dikirim dalam bentuk plaintext sehingga dapat dibaca
4. Pada HTTPS, data terenkripsi sehingga konten tidak dapat dibaca

D. Enkripsi File dengan ZIP Password

1. Buat beberapa file untuk diarsipkan:

```
echo "Dokumen Rahasia 1" > file1.txt  
echo "Dokumen Rahasia 2" > file2.txt
```

2. Buat arsip ZIP terenkripsi dengan password:

```
zip -e rahasia.zip file1.txt file2.txt
```

Masukkan password yang aman saat diminta oleh sistem.

3. Coba ekstrak file tanpa password. Apa yang terjadi?

```
unzip rahasia.zip
```

4. Ekstrak file dengan password yang benar:

```
unzip rahasia.zip # masukkan password saat diminta
```

E. Hasil Praktikum (Dasar)

1. Apa perbedaan antara HTTP dan HTTPS? Mengapa HTTPS lebih aman?

Jawaban:

.....

2. Sertakan screenshot detail sertifikat digital dari sebuah website HTTPS.

Jawaban:

.....

Screenshot:

.....

3. Informasi apa saja yang terdapat dalam sertifikat digital? Jelaskan masing-masing.

Jawaban:

.....

4. Sertakan screenshot capture paket TLS Handshake pada Wireshark.

Jawaban:

.....

Screenshot:

.....

5. Mengapa data pada koneksi HTTPS tidak dapat dibaca oleh Wireshark?

Jawaban:

.....

6. Sertakan screenshot proses enkripsi ZIP.

Jawaban:

.....

Screenshot:

.....

7. Apa yang terjadi jika Anda lupa password ZIP? Mengapa enkripsi password tidak dapat di-bypass?

Jawaban:

.....

8. Sebutkan 5 contoh penerapan kriptografi dalam kehidupan sehari-hari.

Jawaban:

.....

F. Persoalan Sedang — Self-Signed Certificate dan Analisis Kepercayaan Browser

Durasi: 30 menit | Tingkat Kesulitan: Sedang

Buat self-signed certificate menggunakan OpenSSL untuk domain fiktif akademik.local dengan masa berlaku 365 hari. Gunakan Python untuk membuat HTTPS server sederhana di localhost yang menggunakan sertifikat tersebut.

Tugas:

1. Buat self-signed certificate untuk domain akademik.local menggunakan OpenSSL dengan masa berlaku 365 hari.

Jawaban:

.....

2. Gunakan Python untuk membuat HTTPS server sederhana di localhost yang menggunakan sertifikat tersebut (gunakan modul `http.server` dengan SSL wrapper, atau gunakan `openssl s_server`).

Jawaban:

.....

3. Akses `https://localhost` atau `https://akademik.local` melalui browser.

Jawaban:

.....

4. Sertakan screenshot peringatan keamanan ("Your connection is not private" / "Not Secure") yang muncul di browser.

Jawaban:

.....

Screenshot:

.....

5. Jelaskan: Mengapa browser tidak mempercayai self-signed certificate? Apa perbedaan antara self-signed certificate dan certificate yang diterbitkan oleh Certificate Authority (CA) terpercaya seperti Let's Encrypt atau DigiCert? Bagaimana solusi agar self-signed certificate bisa dipercaya dalam lingkungan internal perusahaan (enterprise)?

Jawaban:

.....

Referensi Kode:

```
# 1. Generate self-signed certificate
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem
-days 365 -nodes -subj "/CN=akademik.local"

# 2. Python HTTPS Server
import http.server
import ssl

server_address = ('localhost', 4443)
httpd = http.server.HTTPServer(server_address,
http.server.SimpleHTTPRequestHandler)

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile='cert.pem', keyfile='key.pem')
httpd.socket = context.wrap_socket(httpd.socket, server_side=True)

print("Server HTTPS berjalan di https://localhost:4443")
httpd.serve_forever()

# 3. Alternatif dengan OpenSSL s_server
openssl s_server -cert cert.pem -key key.pem -accept 4443 -www
```

G. Persoalan Susah — Forensik Jaringan: Analisis HTTP vs HTTPS dengan Wireshark dan SSL Key Log

Durasi: 45 menit | Tingkat Kesulitan: Susah

Lakukan analisis forensik jaringan dengan membandingkan HTTP (tidak aman) dan HTTPS (aman), termasuk teknik advanced decryption menggunakan SSL Key Log.

Tugas:

1. Capture HTTP (Tidak Aman): Akses situs HTTP (contoh: <http://neverssl.com> atau buat server HTTP lokal dengan form login). Capture dengan Wireshark, filter dengan http. Temukan dan sertakan screenshot packet yang memperlihatkan password/kredensial dalam plaintext.

Jawaban:

.....

Screenshot:

.....

2. Capture HTTPS (Aman — Default): Akses situs HTTPS populer (Google, GitHub, dsb.). Filter dengan tls di Wireshark. Sertakan screenshot yang menunjukkan bahwa data aplikasi terenkripsi dan tidak bisa dibaca.

Jawaban:

.....

Screenshot:

.....

3. Capture HTTPS dengan Decryption (Advanced): Konfigurasi environment variable SSLKEYLOGFILE di browser (Firefox/Chrome) untuk menyimpan session keys. Akses kembali situs HTTPS. Konfigurasi Wireshark untuk membaca file SSLKEYLOGFILE tersebut (Edit → Preferences → Protocols → TLS → (Pre)-Master-Secret log filename). Sertakan screenshot perbedaan: traffic HTTPS sebelum dan sesudah di-decrypt di Wireshark.

Jawaban:

.....

Screenshot:

.....

4. Analisis Keamanan: Jelaskan risiko keamanan jika file session keys bocor ke pihak ketiga. Jelaskan konsep Perfect Forward Secrecy (PFS) dan bagaimana mekanisme ini (menggunakan algoritma seperti ECDHE) mencegah dekripsi data masa lalu meskipun private key server dicuri di masa depan.

Jawaban:

.....

Referensi Kode & Perintah:

```
# Konfigurasi SSLKEYLOGFILE (Linux/Mac)
export SSLKEYLOGFILE=~/.ssl-keys.log

# Jalankan browser dari terminal yang sama
# Firefox:
firefox &
# Chrome:
google-chrome --ssl-key-log-file=~/.ssl-keys.log &
```

```

# Di Wireshark:
# Edit → Preferences → Protocols → TLS
# (Pre)-Master-Secret log filename: ~/ssl-keys.log

# Setelah konfigurasi, traffic HTTPS akan terdekripsi di Wireshark
# Perhatikan perbedaan pada kolom "Info" dan "Protocol"

# Python HTTP Server sederhana untuk demo plaintext
# server.py
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length)
        print(f"Received: {post_data.decode()}")
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"OK")

httpd = HTTPServer(('localhost', 8080), Handler)
print("HTTP Server running on port 8080")
httpd.serve_forever()

```

H. Kesimpulan

- Tuliskan kesimpulan Anda tentang implementasi kriptografi pada HTTPS.
- Tuliskan kesimpulan Anda tentang penggunaan Wireshark untuk analisis keamanan jaringan.
- Tuliskan kesimpulan Anda tentang enkripsi file menggunakan password.
- Tuliskan pentingnya kriptografi dalam kehidupan digital modern.
- Tuliskan refleksi Anda tentang risiko keamanan yang muncul ketika session keys dapat diekspos dan bagaimana Perfect Forward Secrecy mengatasinya.

Ringkasan Perintah

Berikut adalah ringkasan semua perintah yang digunakan dalam modul praktikum ini sebagai referensi cepat:

Perintah	Fungsi
<code>openssl enc -aes-256-cbc -salt -in [file] -out [file.enc]</code>	Enkripsi file menggunakan AES-256-CBC
<code>openssl enc -d -aes-256-cbc -in [file.enc] -out [file]</code>	Dekripsi file AES-256-CBC
<code>openssl genrsa -out private.pem 2048</code>	Membuat private key RSA 2048-bit
<code>openssl rsa -in private.pem -pubout -out public.pem</code>	Membuat public key dari private key
<code>openssl rsautl -encrypt -pubin -inkey public.pem -in [file] -out [file.enc]</code>	Enkripsi pesan dengan public key RSA
<code>openssl rsautl -decrypt -inkey private.pem -in [file.enc] -out [file]</code>	Dekripsi pesan dengan private key RSA
<code>sha256sum [file]</code>	Membuat hash SHA-256 dari file
<code>openssl dgst -sha256 -sign private.pem -out sign.bin [file]</code>	Membuat digital signature
<code>openssl dgst -sha256 -verify public.pem -signature sign.bin [file]</code>	Verifikasi digital signature
<code>zip -e [output.zip] [input.file]</code>	Membuat arsip ZIP terenkripsi dengan password

Tips Mengerjakan Laporan Praktikum

- 1. Baca pertanyaan dengan teliti.** Pahami apa yang ditanyakan sebelum menjawab. Pastikan jawaban Anda relevan dengan pertanyaan yang diajukan dan mencakup semua aspek yang diminta.
- 2. Praktik terlebih dahulu.** Lakukan semua langkah praktikum secara mandiri sebelum menjawab pertanyaan. Pengalaman langsung akan membantu Anda memahami konsep dengan lebih baik.
- 3. Screenshot sebagai bukti.** Setiap jawaban yang memerlukan screenshot harus disertakan dengan tangkapan layar yang jelas dan dapat dibaca. Pastikan output terminal atau tampilan browser terlihat dengan baik.
- 4. Jawaban singkat tapi tepat.** Berikan jawaban yang akurat, ringkas, dan langsung menjawab inti pertanyaan. Hindari jawaban yang terlalu panjang tapi tidak fokus.
- 5. Verifikasi jawaban.** Periksa kembali semua jawaban Anda sebelum mengumpulkan laporan. Pastikan tidak ada kesalahan pengetikan atau informasi yang kurang lengkap.
- 6. Simpan file hasil praktik.** Lampirkan file-file yang dihasilkan selama praktikum seperti file terenkripsi, kunci, digital signature, dan screenshot sebagai bukti kerja Anda.

Ketentuan Pengumpulan

Format Laporan

- Font: Arial, ukuran 12 pt
- Spasi: 1.5
- Margin: Normal (2.54 cm semua sisi)
- Sertakan screenshot hasil praktikum sebagai bukti
- Dikumpulkan dalam format PDF

Format Nama File

Gunakan format berikut untuk penamaan file laporan:

Nama_NoAK_Laporan_Praktikum_X.pdf

Contoh: Ahmad_01_Laporan_Praktikum_1.pdf

Pengumpulan

Dikumpulkan sesuai petunjuk dosen pengampu mata kuliah. Pastikan laporan dikumpulkan tepat waktu sesuai batas waktu yang telah ditentukan.